Date
Page

ONE SHOT
UNIT-1 (C.O.A)
INTRODUCTION

SUBSCRIBE CHANNEL

JOIN TELEGRAM
MULTI ATOMS

A.K.T.U

Date
Page

① Topics:-

⇒ Functional Units of Digital System
⇒ Bus, Bus Architecture, Type of Buses
⇒ Bus Arbitration
⇒ Three state Bus
⇒ Register stack Vs Memory Stack
⇒ Addressing Modes.

① **Computer**: A computer is an electronic device that takes input from the user, processes it under the control of set of instruction and generates output.
→ It has the ability to store, retrieve, and process data.

→ A computer is made up of multiple components that facilitates user functionality.
→ A computer has two primary category.

② **Hardware**: Physical structure that houses a computer's processor, memory, storage, communication ports and peripheral devices.

③ **Software**: Includes operating system (OS) and software applications.

Date
Page

Computer Architecture and Computer Organization AKTU 2019-20 2020-21

Computer Architecture

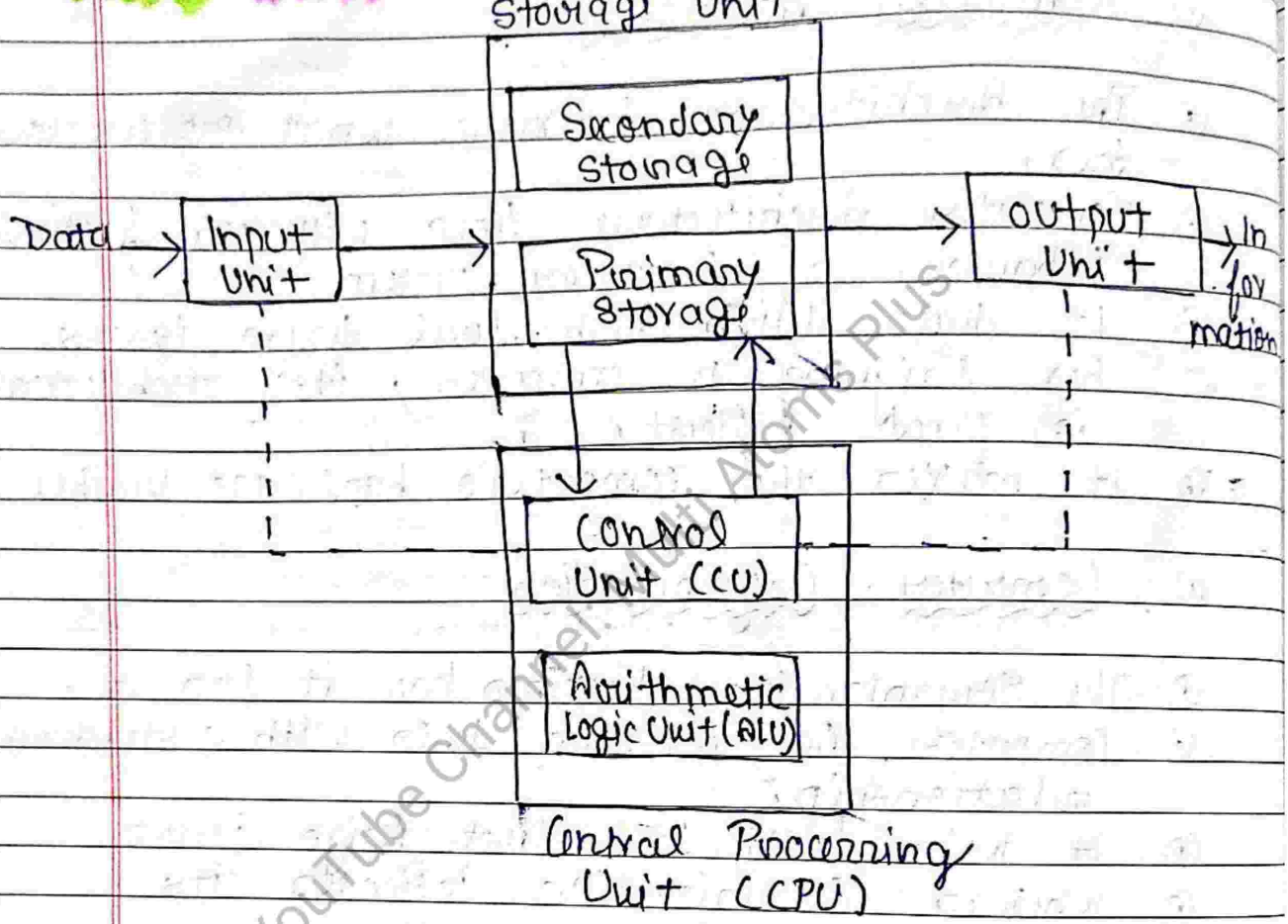
- The Architecture describes what the computer does.
- Computer Architecture deals with the functional behaviour of computer system.
- It deals with high-level design issues.
- For designing a computer, its architecture is fixed first.
- It makes the computer's hardware visible.

Computer Organization

- The Organization describes how it does it.
- Computer Organization deals with a structural relationship.
- It deals with low-level design issues.
- Whereas Organization indicates its performance.
- For designing a computer, an organization is decided after its architecture.
- It offers details on how well the computer performs.

① Functional Unit of Digital System:-

AKTU 2022-23, 2019-20



1. **Input Unit**: We need to first enter the data & instruction in the computer system, before any computation begins.
→ This task is done by the input devices (E.g keyboard, mouse, scanner, etc).

2. **Storage Unit**: The data & instruction that are entered have to be stored in the computer
→ Similarly, the end result & the intermediate result also have to be stored somewhere before being passed to the output unit.

→ The storage unit provide solution to all these issues
→ A storage unit holds stores.

(i) Primary Storage: It is also called main memory of computer.
→ It is computer memory that a processor or computer accesses first or directly.
→ It allows a processor to access running execution applications and services that are temporarily stored in a specific memory location
→ It is of two types RAM and ROM.

ii) Secondary Storage :- The secondary storage, also called as the auxiliary storage, can retain information even when the system is off.
→ It is basically used for holding the program instructions & data on which the computer not working on currently, but needs to process them later

3. Central Processing Unit: CPU has two major components: ALU (Arithmetic logic unit) and CU (Control Unit).
→ The CPU is the brain of the computer.
→ In a computer, all the major calculation, & comparisons are made inside the CPU.

(i) Arithmetic logic Unit: The actual execution of the instructions (arithmetic or logical operation) take place over here.

→ The ALU performs simple addition, subtraction, multiplication, division, and logic operations. such OR and AND.
→ Intermediate results that are generated in ALU are temporarily transferred back to the primary storage, until needed later.
→ Hence, data may move from the primary storage to ALU & back again to storage, many times, before the processing is done

(ii) Control Unit: This unit controls the operations of all parts of the computer but does not carry out any actual data processing.
→ It manages and coordinates all the unit of the system.
→ It also communicates with input/output device for transfer of data or result from the storage units

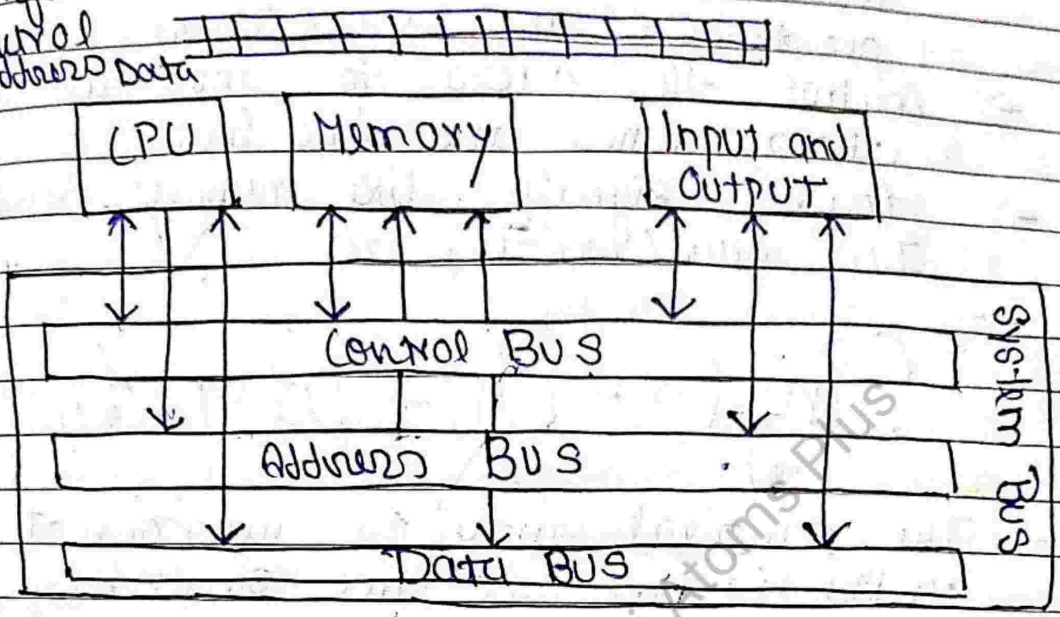
4. Output Unit:- Output unit accepts the result produced by the computer in coded form
→ It convert these code results to Human readable form.
→ Finally, it displays the converted result to the outside world with the output device. (E.g monitors, printers, projectors etc).

⊙ Bus:- AKTU 2019-20, 2021-22, 23

⊙ Bus is a group of wires (or lines) that connects several devices with a computer system
⊙ Each wire/line can transfer one bit (1/0) of information.

Date
Page

① Bus carries data, address and control information



⇒ Types of Buses :-

⇒ Address Bus : It is a group of wires which carries address information bits from processor to peripherals.

⇒ Address Bus width determines the maximum memory capacity.

⇒ In general $2^x = n$ where x number of address lines (address bit) and n is number of location.

⇒ Address Bus is Unidirectional.

⇒ Data Bus : It is a group of wires which carries data information bit from processor to peripherals and vice - versa.

⇒ Move data / instruction between CPU and peripheral

⇒ Data bus width determines the system performance.

⇒ Data Bus is bidirectional.

⇒ Control bus : It is a group of wires which carries control signals from processor to peripherals and vice-versa.

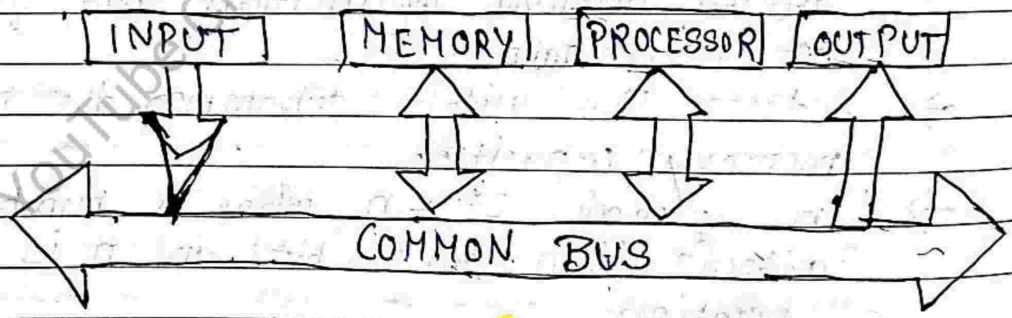
→ Control the access to and the use of address lines and data lines

→ Control signals like memory read/write, I/O read/write, etc.

⇒ Single-Bus Structure :

→ The simplest way to interconnect functional units is to use the single bus.

⇒ Single bus structure :- Common bus used to communicate between peripherals and microprocessor.



Single BUS Structure

⇒ Single bus does one transfer at a time, so only 2 units can actively use the bus at a given time.

⇒ Bus control lines are used to arbitrate multiple requests for use of one bus

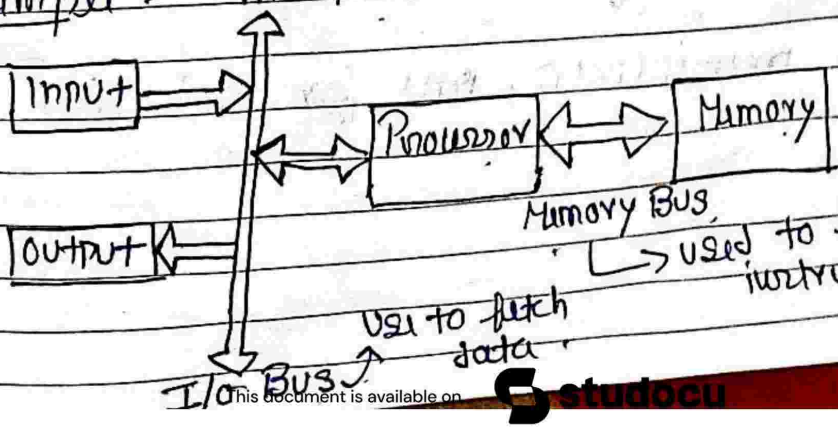
Advantage :-
- Simple and low cost
- Very flexible for attaching peripheral devices

Drawback :-
- Speed issues : Device connected to the bus vary widely in their speed of operation
- Efficient transfer mechanism is needed to cope with this problem like buffer register, Multiple Bus structure

⇒ Multi-Bus Structure

⇒ System that contain multiple buses achieve more parallelism.
⇒ This leads to better performance but ant an increase cost.
⇒ Multi-bus structure improves the performance but also increases cost significantly.
⇒ In Multi-bus structure, each of which connects subset of modules

Example :- In two-bus structure.



Advantage! -
-> It improves Efficiency
-> It improves performance

Disadvantage! -
-> costly

Numerical : AKTU 2021-22

Q A digital computer has a common bus system for 8 registers of 16 bit each. The bus is constructed using multiplexers.

(i) How many select input are there in each multiplexers.
(ii) What is the size of multiplexers needed?
(iii) How many multiplexer are there in the bus?

Sol (i) Each multiplexer must have 8 data input lins and three selection lines (S_2, S_1, S_0) to multiplex one significant bit in the eight register.

no. of register $= 8 = 2^n = 2^3$

where, n is the number of selection line.

(ii) So size of MUX is 8X1

(iii) 16 multiplexer, one for each line in the bus

⇒ Bus Arbitration. AKTU 20-21

① Bus Arbitration refers to the process by which the current bus master accesses and then bus and passes it to another bus requesting processor unit.

② The controller that has access to a bus at an instance is known as a Bus master.

⇒ There are two approaches to bus arbitration:

1. Centralized bus arbitration –
   A single bus arbiter performs the required arbitration.

2. Distributed bus arbitration – All device participating in the selection of the next bus master.

⇒ Methods of Centralized Bus Arbitration:-

(i) Daisy chaining method:-

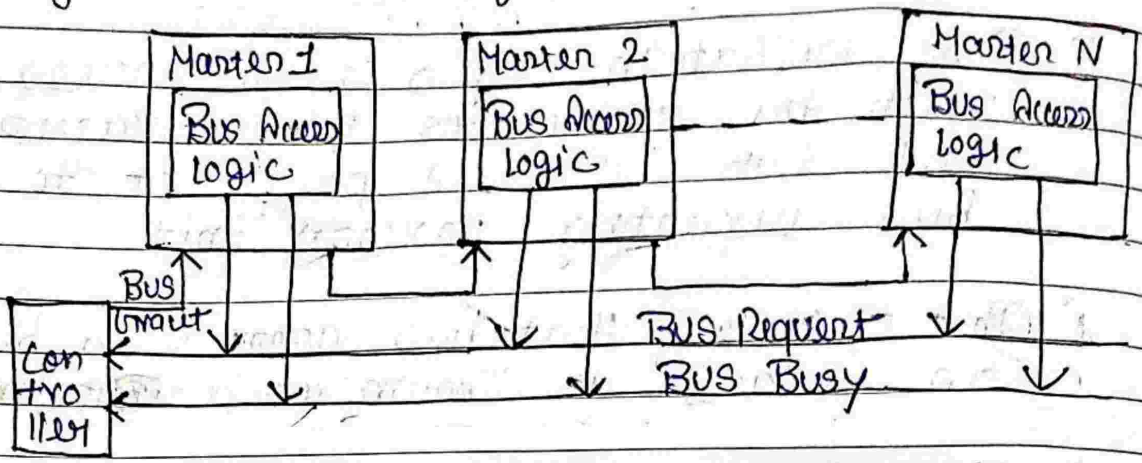
⇒ Daisy chaining method is cheaper and simple method.

⇒ All master make use of the same line for bus request.

⇒ The bus grant signal serially propagates through each master until it encounters the first one that is requesting access to the bus.

⇒ This master blocks the propagation of the

grant signal, activates the busy line and gains control of the bus.



Adv :-

- It is simple and cheaper method.
- Simplicity and Scalability

Disadv :-

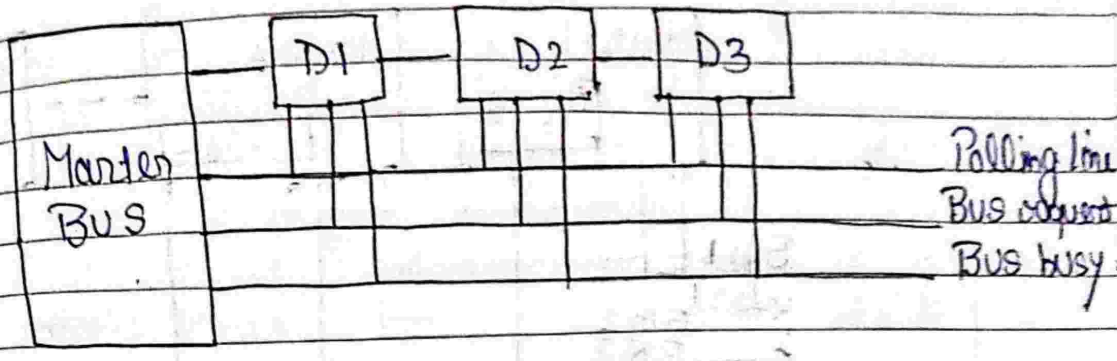
- The value of priority arrigned to a device depends on the position of the master bus
- If one device fails then the entire system will stop working.

(ii) Polling :-

→ In this method, a controller generates unique address for each master (device) based on their priority.

→ The number of address line corrulates with the number of masters in the system

→ The controller cycles through the generated addresses. When master recognizes its own address, it activates a "busy" signal and gain access to the bus for data transfer



Adv :-

① This method does not favour any particular device and processor

② The method is also quite simple.

Disadv :

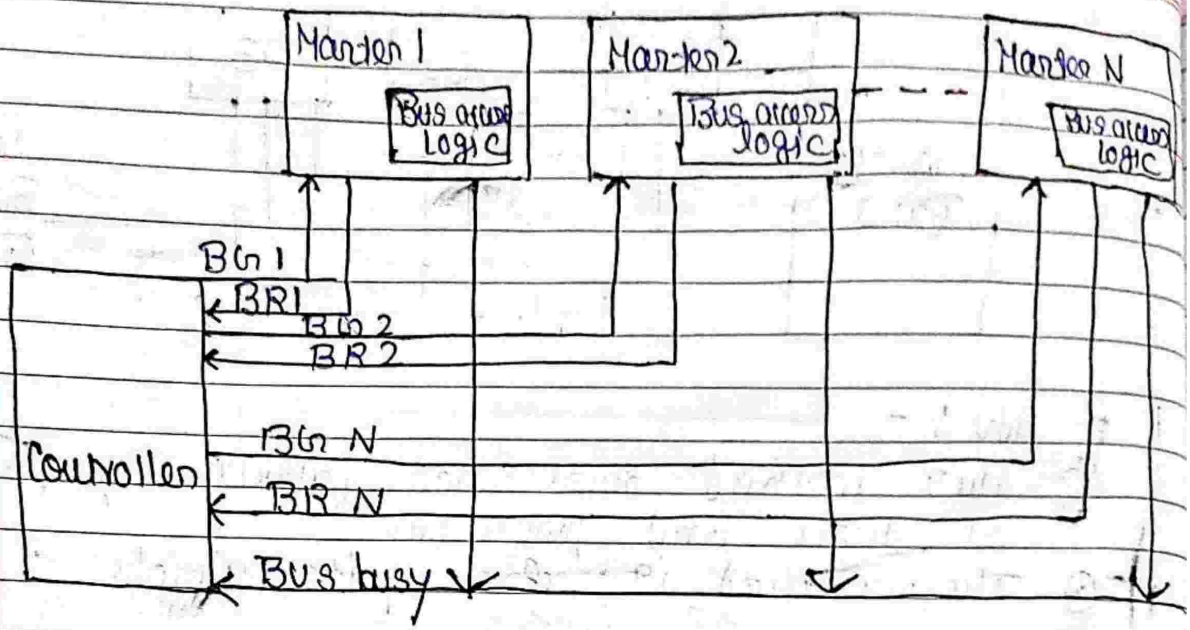
① Adding Bus master is difficult as increases the number of address line of the circuit.

② If one device fails then entire system will not stop working

iii) Fixed priority or Independent Request Method :-

-> In this, each master has a siparate pair of bus request and bus grant lines and each pair has a priority assigned to it.

-> The built-in priority decoder within the controller selects the highest priority request and asserts the corresponding bus grant signal.



where BG → Bus Grant
BR → Bus Request.

Adv :-
→ This method generates a fast response

Disadv:
→ Hardware cost is high as a large no. of control is required.

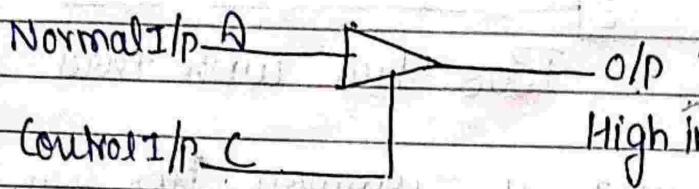
⇒ THREE-STATE BUS BUFFER. AKTU-20-21, 21-22

→ A bus system can be constructed with three state gates instead of multiplexer, is known as three state bus system

→ A three-state gate is a digital circuit that exhibits three states

→ Two of the state are signals equivalent to logic 1 and 0 as in a conventional gate.

-> The third state is a high-impedance state.
-> The high-impedance state behaves like an open-circuit, which means that the O/P is disconnected and does not have a logic significance.
-> Three state gates may perform any conventional logic, such as AND or NAND

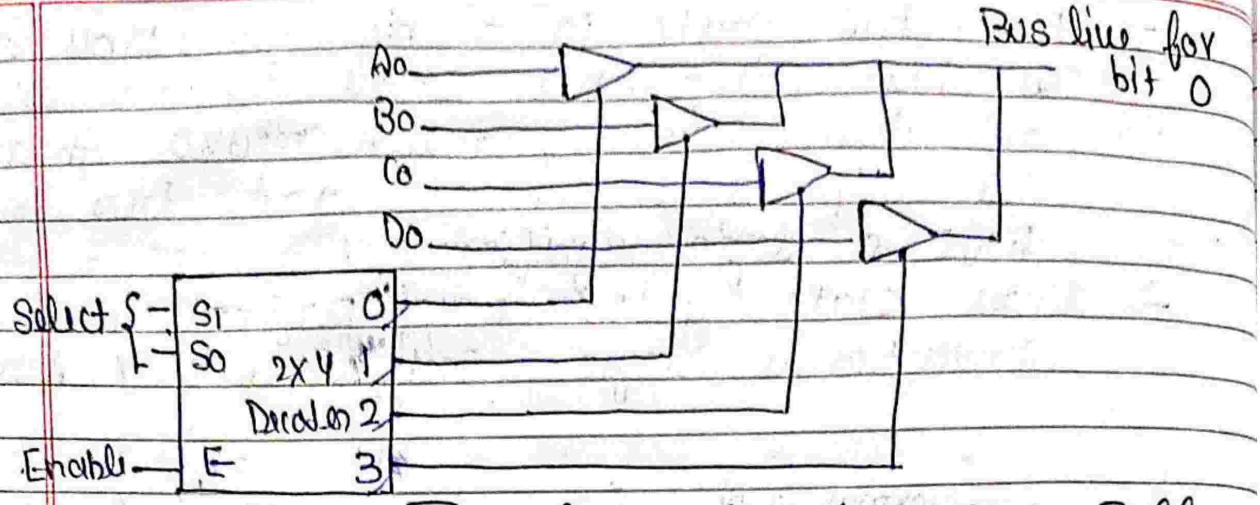


Graphical symbol of three state buffer

① The control I/p determines the output state.
① When the control input is equal to 1, output is enabled and the gate behaves like any conventional buffer.

① When the control input is 0, the output is disabled and the gate goes to a high state, regardless of the value is the normal input

① Because of this high-impedance state, a large number of three-gate O/p can be connected with wired to form a common bus line without endangering loading effect.

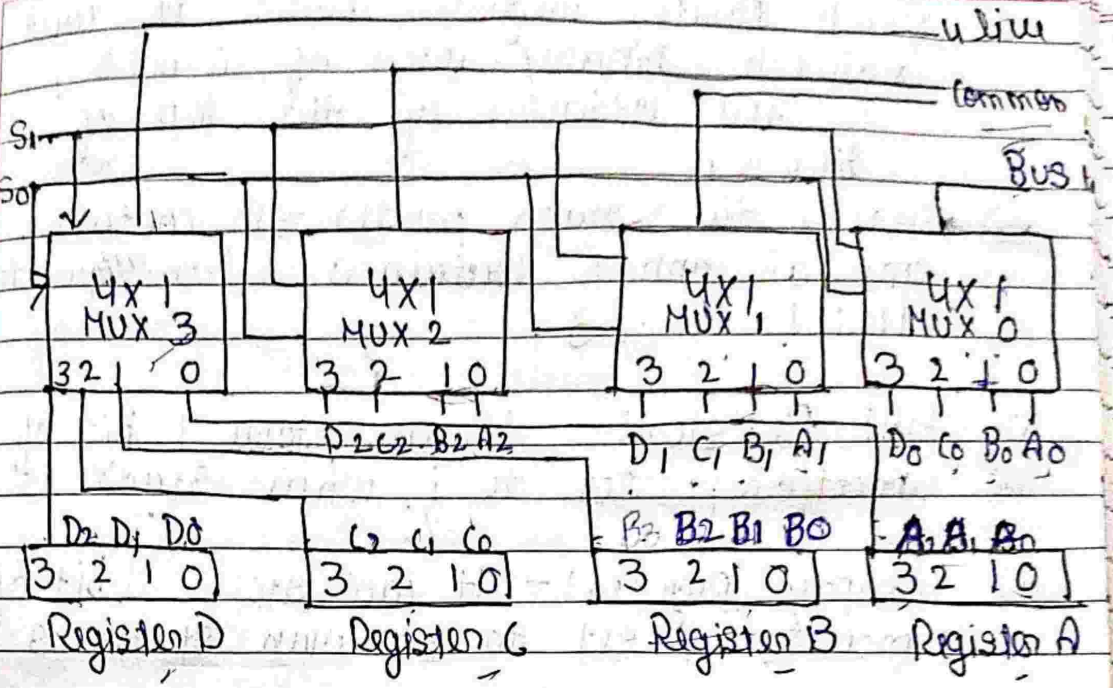


Bus line with three state Buffers

- To construct a common bus for four registers of n bits each using three state buffers, we need n circuits with four buffers
- Each group of four buffer receive one significant bit from the four registers.
- Each common o/p produces one of the lines for the common bus for total of n lines
- Only one decoder is necessary to select between the four register.

AKTU 20-21

Q Draw a diagram of bus system using MUX which has four register of size 4-bits each:



⇒ STACK ORGANIZATION

AKTU 2021-22
2020-21, 2022-23

① It is an ordered list in which addition of new data item and deletion of already existing data item is done from only one end known as top of stack (TOS)

② The element which is added in last will be first to be removed and the element which is inserted first will be removed in last, so it is called last in first out (LIFO) or First in last out (FILO).

③ Most frequently accessible element in the stack is the topmost element, whereas the least accessible element is the bottom of the stack.

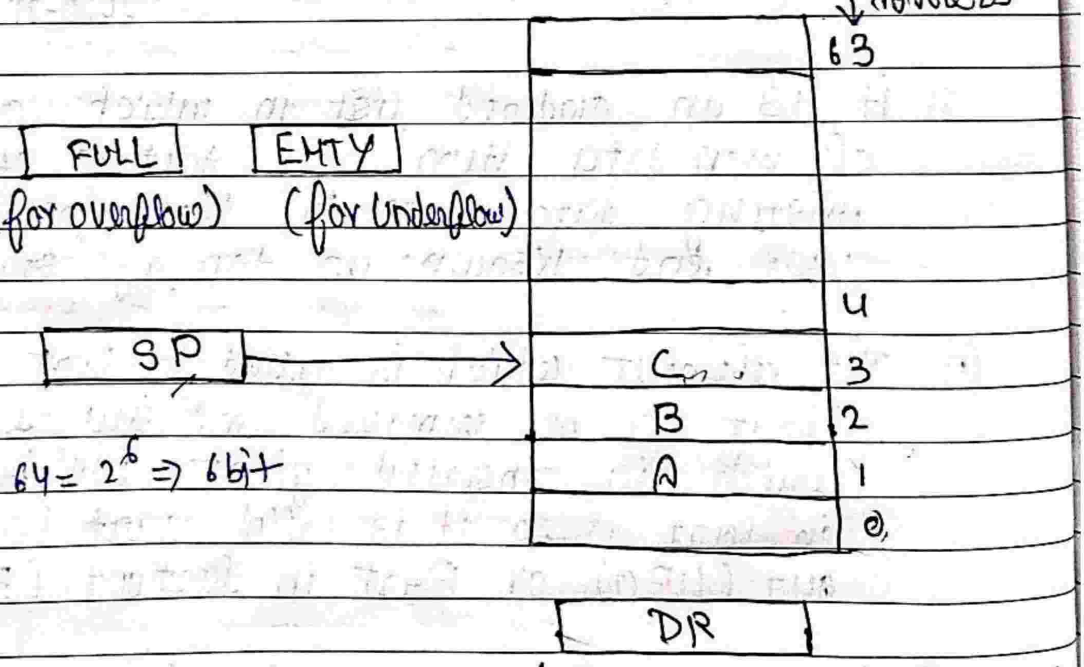
① Stack Pointer register (SP): It contains a value in binary each of 6 bits, which is the address of the top of the stack.

→ Here, the stack pointer SP contains 6 bits and SP cannot contain a value greater than 111111 i.e. 63

① Full Register:- It can store 1 bit of information, set to 1 when stack is full

① Empty Register:- It can store 1 bit of information, set to 1 when stack is empty

① Data Register:- It hold the data to be written into be read from the stack



(Something is Pop and push to/from the stack is in the DR)

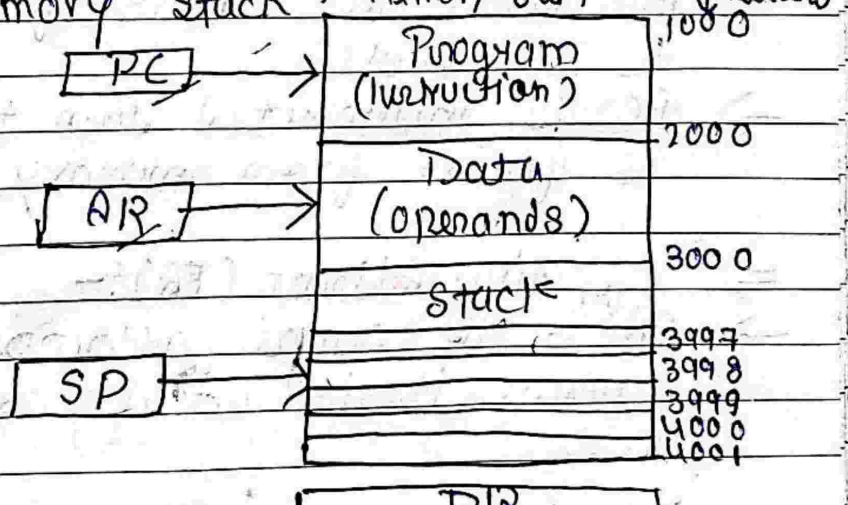
⇒ MEMORY STACK AKTU 20-21, 23, 22

→ Memory Stack operate on a last-In, first-Out (LIFO) principle, where the most recently added element is the first to be removed.

→ A special register called the stack pointer (SP) points to the top of the stack, and it can contains binary values up to a limit (often determined by the architecture, such as 6 bits equating to 63 in decimal).

→ To monitor the stack's status, Full and Empty Registers are used, each storing a single bit to indicate whether the stack is full or empty.

→ A Data Register holds the data to be written into or read from the stack serving as an intermediary between the CPU and the stack. These elements collectively form the essential organization of a memory stack.



ADDRESSING MODES AKTU 2021-22, 2022-23, 20-21

⇒ Address modes :- The different ways of specifying the location of an operand in an instruction are called as addressing modes.

☆ The purpose of using addressing modes is :
→ To give the programming visibility to the user.
→ To reduce the number of bits in addressing field of instruction

⇒ Instruction cycle :-
→ Fetch the instruction from memory and PC←PC+
→ Decode the instruction
→ Execute the instruction

⇒ Program counter
→ PC keeps track of the instructions in the progra stored in memory
→ PC holds the address of the next instructi to be executed
→ PC is incremented each time an instruction is fetched from memory

⇒ Effective Address (EA) :-
→ EA is the actual address of the operand where it is actually stored.

2024 Vdtu

TYPES OF ADDRESSING MODES

AKTU 21-22, 22-23

1. Implied / Implicit Mode } No address field is required.
2. Immediate Mode
3. Direct Mode } Address specifies memory location
4. Indirect Mode
5. Register Direct Mode } - Address specifies Processor register
6. Register Indirect Mode
7. Auto-increment / Auto-Decrement Mode
8. Relative Addressing Mode } - Address field + content of CPU register.
9. Indexed Addressing Mode
10. Base Register Addressing Mode

1. Implied (or Implicit) Mode

=> Operand are specified implicitly in the definition of instruction.

[Data]

Example:-

CLC (used to reset carry flag to 0)

2. Immediate Mode.

=> Operand is specified in the instruction itself.

=> An immediate mode instruction has an operand field rather than the address field.

=> Operand field contain the actual operand

=> Operand = Address

Instruction

Opcode	Operand

- Example
```
LOAD #7        // AC ← 7
ADD #5         // AC ← AC+5
```

→ It is useful for initiating register to a constant value.

3. Direct Mode.

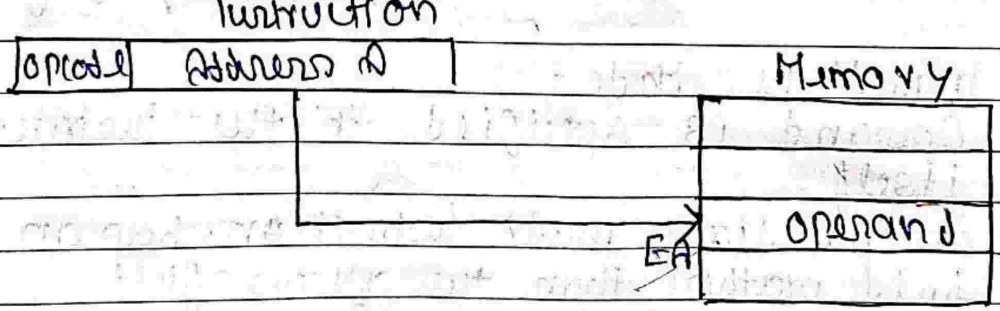
→ The operand resides in memory and its address is given directly by the address field of instruction

→ Address field of instruction contains the effective address (EA) of operand.

→ EA = A Effective address (EA) is equal to the address field (A).

① Example
```
ADD A          // AC ← AC + M[A]
LOAD B         // AC ← M[B]
```

→ It is also called an absolute addressing mode

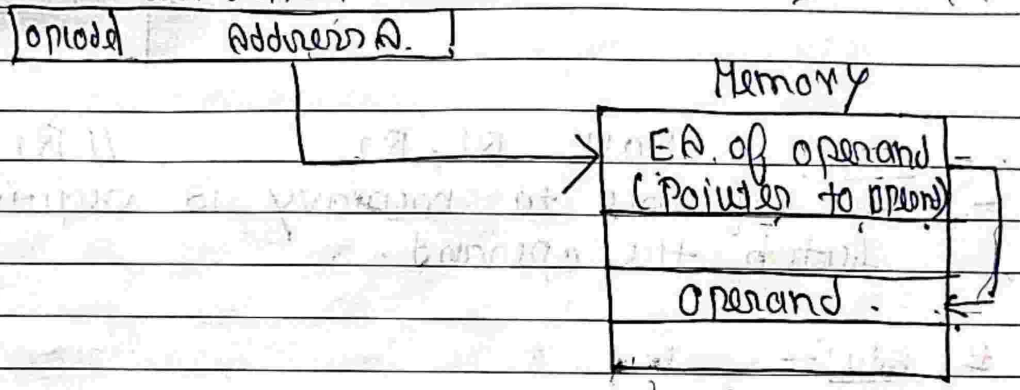


Adv :-
- Simple
- No additional calculations to work on EA.

Disadv :
-> Address space is limited to the size of the address field.

4. Indirect Mode
-> Address field of instruction specifies the address where the effective address of operand is stored in memory

-> EA = (A) or EA = @A Look in A, fin add(A) and look there for operand



① Example ADD (A) // AC ← AC + M[M[A]]

Adv :-
① Larger address space is possible
① 2^n where n = word length

Disadv :-
① Multiple (two) memory reference to find the operand
① Hence slower.

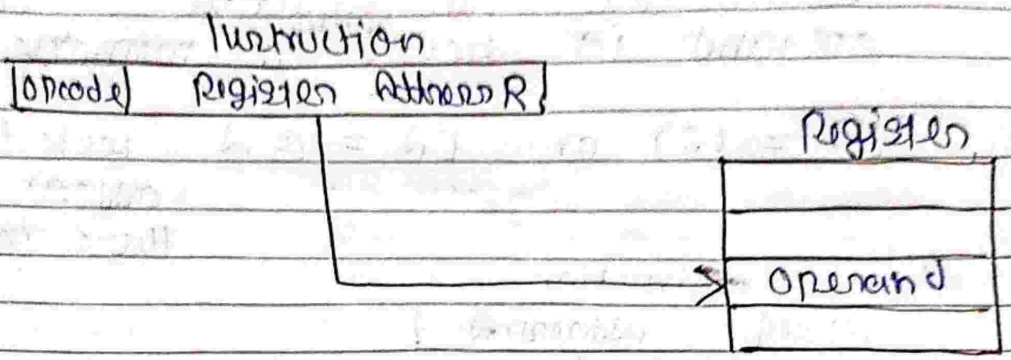
5. Register Mode.

→ Operand is stored in the register

→ Address field of the instruction refers to a CPU register that contains the operand.

→ EA = Ri

→ A k-bit field can specify any one 2^k registers



Ex: MOV R1, R2 // R1 ← R2

→ No reference to memory is required to fetch the operand.

Adv:-

→ Shorter instruction

→ Very fast execution (No memory reference)

Disadv:-

→ Very limited address space.

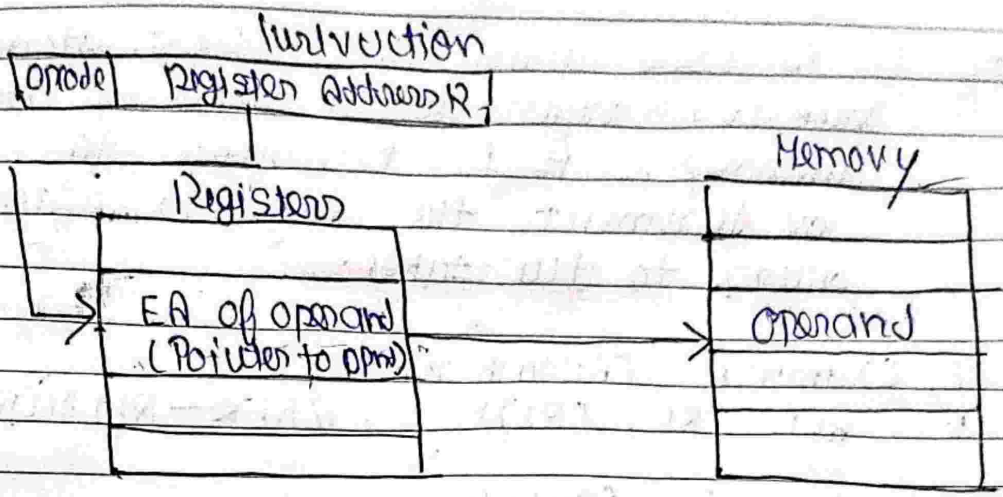
6. Register Indirect Mode:-

→ The address field of instruction refers the register that contains the effective address of operand.

→ The operand is in memory

→ The register contains the effective address of operand rather than the operand itself.

① EA = (Ri)



Ex :- ADD (R1), R2 //M[R1] = M[R1]+R2

Adv :-

① Address field of instruction uses fewer bits to select a memory address

② Large address space : 2^n

Disadv :-

① Extra memory reference.

7. Auto - increment / Auto - decrement Mode

- Similar to register indirect mode (EA of operand is the content of register specified in the instruction) except that

① In Auto-increment : The register is incremented after its value is used to access memory

EA = (Ri)+

Date
Page

→ In Auto decrement :- The register is decrement before its value is used to access memory.

EA = -(Ri)

→ It is used when the address stored in the register refers to a table of data in memory, it is necessary to increment or decrement the register after every access to the table.

// Example (Auto increment)
i. ADD R1, (R2)+ // R1 ← R1 + M[R2], R2 ← R2+1

// Example (Auto decrement)
i ADD R1, -(R2) // R2 ← R2 - 1, R1 ← R1 + M[R2]

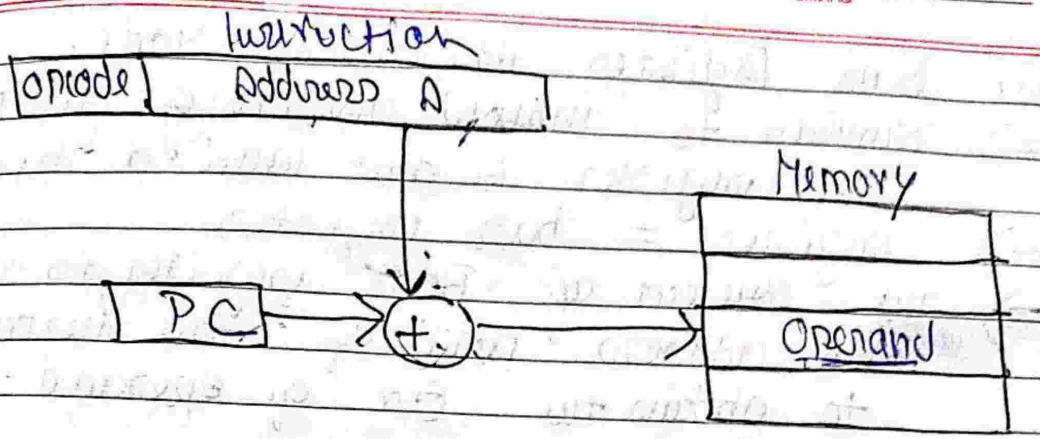
8. Relative Addressing Mode
⇒ Register = Program Counter (PC).
→ The content of PC is added to the address part of the instruction to obtain the E.A of operand.

E.A = Content of PC + Address part of instruction

① EA = (PC) + A i.e the operand is A bytes away from current location pointed to by PC.

Ex:- LOAD $A or LOAD A(PC)
// AC ← M[PC + A]

⇒ Address location is relative to "PC" this is called Relative Mode.



7. Indexed Addressing Mode
=> Register = Index Register.
-> The content of Index Register is added to the address part of the instruction to obtain the E.A of operand

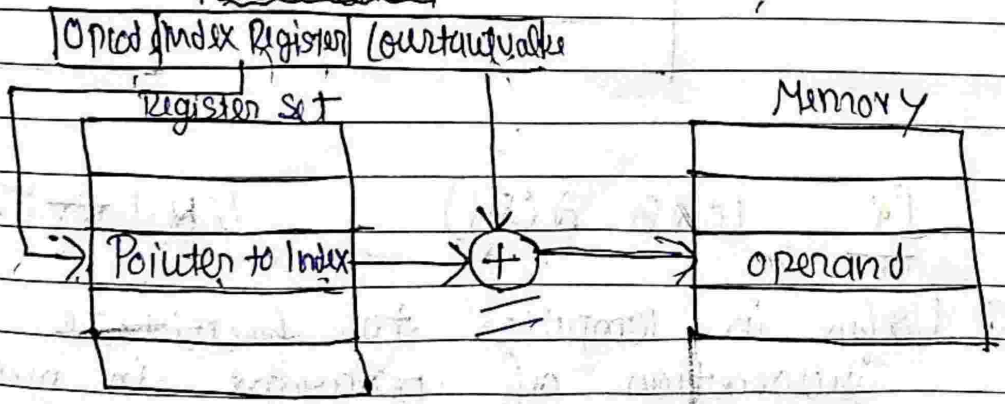
E.A = Content of XR + Address part of instruction

-> E.A = A + (XR)

-> A contains a constant value in instr.
-> XR is the name of register involved holds index value

Example

ADD R1, 20(R2) // R1 ← R1 + M[20+R2]



10. Base Register Addressing Mode

⇒ Similar to indexed addressing except that the register is now called a base register

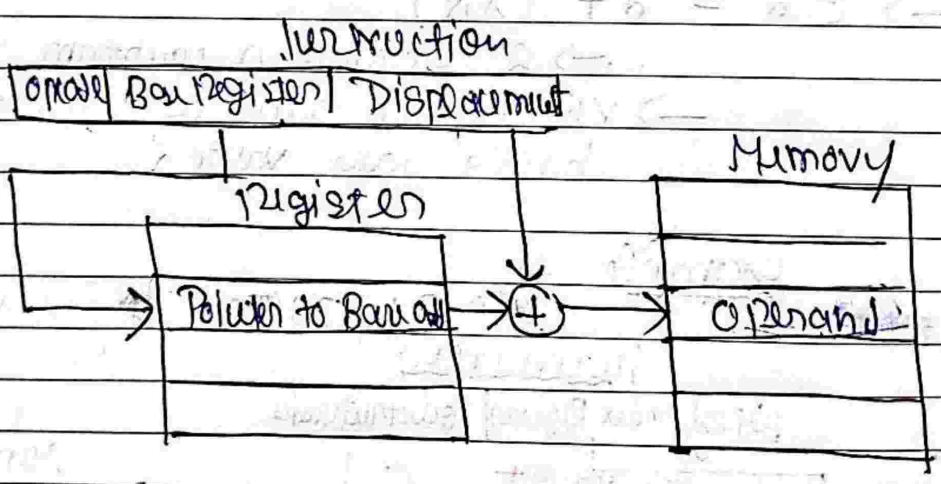
→ register = base register

→ The content of Base register is added to the address part of the instruction to obtain the EA of operand.

EA = Content of BRT Address part of instruction.

EA = (Ri) + A

- Where Ri is a base register that holds base address
- A holds a displacement relative to this base address



Eg LOAD A(R1) // A.1 ← M[R1 + A]

① Useful in computer to facilitate the relocation of program in memory